

The logging systems

IACE platform · what each mechanism is for, what it costs, and what each environment sees · 1 October 2026

There are **eight** mechanisms that record something. Only three of them write to stdout, and one variable — `LOG_LEVEL` — decides how much of that a given environment hears. The other five do not touch stdout at all, which is why they were easy to mistake for one tangled system. This is the role of each, in the order a request meets them.

1 · The map

MECHANISM	WRITES TO	ITS ONE JOB	LIVES FOR
<code>logging.ts</code> NEW	—	Decides stdout's <i>level</i> and <i>format</i> . The control plane, not a producer.	n/a
<code>RequestObserver</code> middleware	stdout DEBUG	Counts <i>and</i> logs EVERY request — including the ones a guard refused before any interceptor ran.	10 MB × 3 roll
<code>AllExceptionsFilter</code>	stdout DEBUG/ERROR	Says a call <i>failed</i> ; a 5xx carries the stack, the redacted body and a bug= fingerprint.	10 MB × 3 roll
76 domain statements	stdout WARN/ERROR	Says something <i>degraded</i> or that bookkeeping was lost.	10 MB × 3 roll
<code>redact.ts</code>	—	The PII boundary: logged bodies, masked mobiles, push-endpoint hosts. A filter, not a producer.	n/a
<code>/metrics</code>	Prometheus SCRAPE	Numbers over time. Never parsed from a log — counted at the source.	Grafana
<code>RowActionLog</code>	Postgres TABLE	Who changed which record, and to what. Queryable, exportable, permanent.	30d → S3
<code>ImportLog</code>	Postgres TABLE	One row per import run, plus the uploaded file itself.	permanent
Caddy access log	EBS volume	Every HTTP request that reached the box, including ones the API never saw.	100 MiB × 10
BullMQ job records	Redis	The dead-letter queue: a job that exhausted its retries, with its payload.	7d / 500

The single most useful distinction. A *log* is a line you read when something went wrong and then throw away. An *audit row* is a record you query months later and must never lose. This repo has both, and they are not alternatives — `RowActionLog` is why the stdout logs can be allowed to roll away every few minutes without anyone losing anything that matters.

2 · The control plane

`apps/api/src/common/logging.ts` **NEW**

The only place `stdout`'s shape is chosen. It runs before the config module exists, so it reads `process.env` directly — the same thing `main.ts` already does for `UV_THREADPOOL_SIZE`.

```
export function appLogger(): ConsoleLogger {
  const production = process.env.NODE_ENV === NODE_ENVS.PRODUCTION;
  const chosen = process.env.LOG_LEVEL?.trim() ||
    (production ? '>=log' : '>=debug');

  return new ConsoleLogger({ json: production, logLevels: filterLogLevels(chosen) });
}
```

ROLE	Two decisions, nothing else: which levels reach stdout, and whether each line is prose or JSON.
WHY HERE	Before this existed, <code>NestFactory.create</code> got no <code>logger</code> option at all, so Nest's default printed every level in production — including the debug lines written as development aids.
NO DEPENDENCY	<code>ConsoleLogger</code> and <code>filterLogLevels</code> ship in <code>@nestjs/common</code> . Winston or pino would add a dependency to do what the framework already does.
FORMAT	<code>json: true</code> in production only. Development keeps the readable coloured output; a shipper needs one object per line.

What each environment hears

RECORD	LEVEL	DEV / STAGING >=DEBUG	PRODUCTION >=LOG
Every successful call	debug	✓ every one	silent — this is the flood that was filling stdout
A success over 1,000 ms	warn	✓	✓ — the only thing production hears about a call that worked
A 4xx	debug	✓ with its error code	silent — <code>fieldErrors</code> already told the caller
A 5xx	error	✓ stack + body	✓ stack + body
Degradation warnings	warn	✓	✓
Slow query (≥500 ms)	warn	✓	✓
Boot / connection lines	log	✓	✓ (once each)

To debug a live box: set `LOG_LEVEL=>=debug`, restart, reproduce, put it back. Do not leave it on through an event — at 3K concurrent students the per-request line alone is ~150 lines a second, which rolls the 10 MB cap in under two minutes and buries every warning in it.

3 · The three stdout producers

Per-request line — `common/metrics/metrics.interceptor.ts`

The one observer on every request. It already held the timer for the latency histogram, so the log line reuses it rather than starting a second one.

```
GET /me/tests/:id → 200 37ms STUDENT:9f3c... [b41e-...]
```

ROLE	The record that a call happened at all, and how long it took — the only mechanism that says anything about a <i>successful</i> request.
ROUTE PATTERN	<code>/me/attempts/:id</code> , never the resolved URL. Same reason the metric does it: 5,000 attempt ids would be 5,000 Prometheus series, and 5,000 unjoinable log shapes.
TWO LEVELS	<code>debug</code> normally; <code>warn</code> with a <code>SLOW</code> marker past 1,000 ms. That split is what lets one variable turn the flood off while keeping the signal.
MIDDLEWARE, NOT AN INTERCEPTOR	Guards run BEFORE interceptors, so a 401, 403 or 429 never reaches one, and a 413 is thrown before a route is chosen. All of them were missing from the latency metric entirely until this moved to <code>response.on('finish')</code> , which fires whatever produced the response.
ACTOR	<code>STUDENT:<id></code> , <code>ADMIN:<id></code> or <code>anonymous</code> . This is what Caddy's access log cannot tell you.

Failures — `common/all-exceptions.filter.ts`

`@Catch()` with no argument: the single exit for everything thrown anywhere — controllers, guards, pipes, Zod, Prisma, a stray `TypeError` , an Express middleware error.

ROLE	Turn any thrown value into exactly one envelope error, and leave behind the evidence the response refuses to carry.
4XX	One <code>debug</code> line: route, status, code, request id, the user-facing message. No body — <code>fieldErrors</code> already names every field the caller got wrong.
5XX	<code>error</code> with the stack and the redacted request body , and a <code>bug=</code> fingerprint. The response says only “Something went wrong” on purpose, because an unhandled error's message is as likely to be a connection string as anything useful — so this line is the sole copy of the cause.
NEVER SUPPRESSED	<code>error</code> is above every threshold, so no <code>LOG_LEVEL</code> setting can hide a 500.

73 domain statements — across 41 loggers

Not noise, and not uniform. They fall into five roles worth telling apart.

ROLE	WHAT IT MEANS WHEN YOU SEE ONE
Boot risk main.ts, api-role.ts	A container is misconfigured in a way that is otherwise invisible: an undersized thread pool reads as latency, a heap above the container limit arrives as <code>exit 137</code> with no stack at all.
Dependency ready prisma, redis, storage, mail	Boot-time, once each. The storage line prints the resolved endpoint, so a box accidentally pointed at MinIO is obvious in one glance.
Degraded, continuing throttler, storage, otp, metrics	A non-essential failed and the request was served anyway — the rate limiter failing open, an OTP falling back from WhatsApp to SMS. Nothing else will ever tell you this happened.
Write kept, bookkeeping lost scoring, rollup, audit, listeners	The most important category. The user's action succeeded and a derived count did not: <i>"re-scored but not counted"</i> , <i>"voided but not recounted"</i> , <i>"audit write failed"</i> . There is no metric for these. Reading them is the only way to know.
Bounded work gave up pruneInPages, audit archive	A job hit its per-run ceiling on purpose and will continue next run. Says "backlog", not "bug".

One slow-query log, and why it logs no parameters

`PrismaService` subscribes to Prisma's `query` event and logs only those at or over **500 ms**. The threshold is the volume control — every query would be per-request logging on the hot path. The *parameters are deliberately excluded*: they carry student data. You get the statement shape, never the values.

4 · The six that never touch stdout

`redact.ts` — the PII boundary

Not a producer. Everything PII-shaped that could reach a log passes through it first, and there is exactly one of it.

WHY IT MUST EXIST	<code>instrument.ts</code> sets <code>sendDefaultPii: false</code> precisely so a mobile number or a PIN cannot leave inside a stack frame. Attaching a body by hand would have silently undone that.
KEY MATCHING	Whole key first, so an acronym like <code>PIN</code> matches; then per word from a camelCase/snake_case split, so <code>newPin</code> , <code>mobileNumber</code> and <code>student_email</code> match while <code>expandedPanel</code> does not — <code>panel</code> is not <code>pan</code> .
THREE BOUNDS	2 KB stringified, 6 levels deep, 20 array items. The depth cap is a trust-boundary guard; the size cap is applied once at the top, because a per-level check would restringify every branch it walked.
ALSO THE STRINGS	<code>maskedMobile</code> keeps a number's last four digits — enough to find the row, not enough to be the number. <code>endpointHost</code> keeps a push endpoint's vendor and drops the path, which is the device's send credential, not a name.

Sentry — removed 1 October 2026 GONE

Listed here because it was in this document a day ago. It was wired, correct, and **never switched on** — no DSN was set in any environment — so removing it deleted a mechanism rather than adding one.

WHAT IT DID	Grouped unhandled exceptions by stack, so one bug firing 4,000 times read as one issue rather than 4,000 lines.
WHAT REPLACED IT	The <code>bug=<10 hex></code> fingerprint on every 5xx line. <code>sum by (bug)</code> in a log query is the same grouping.
WHAT IS GENUINELY LOST	Issue state (resolved / ignored / assigned) and release-regression detection. Neither is worth a second vendor and a second alerting config at one student pre-launch.
WHEN TO REVISIT	When enough users exist that the same bug firing thousands of times would drown everything else in the log.

This is the answer to “how do I get metrics from my logs”: **you do not**. Nothing parses a log line to produce a number. Every value is counted at the source, in memory, and scraped — which is why the logs are free to roll away.

Pushed as things happen

`http_request_duration_seconds` (histogram, buckets 0.01→10 *seconds*, because everything downstream of a scrape assumes seconds) · `attempt_submits_total` · `otp_sends_total` · `queue_failures_total` · `queue_connection_errors_total` · `scoring_backlog`.

Pulled on the scrape itself

`queue_depth` and `queue_oldest_waiting_seconds` — depth says how many, oldest-wait says how bad · `redis_memory`, `redis_memory_ratio`, `redis_evictions` from INFO · `live_attempts` from SCARD on the dirty set, i.e. sittings in Redis not yet in Postgres · `db_connections` from `pg_stat_activity`.

SUBTLETY	<code>scoring_backlog</code> exists because queue depth reads zero once a spent job is removed. A reconciler pushes its own count as the honest stand-in.
GUARDED	Bearer <code>METRICS_TOKEN</code> , required in production — live attempt counts and error rates are operational intelligence. <code>@Res()</code> because Prometheus parses line-oriented text and the success envelope is not that.
READ BY	Grafana Cloud's free tier. Also where every alarm threshold lives.

The audit trail — RowActionLog POSTGRES → S3

The strongest record in the system, and the one most easily mistaken for a log. Opt in per route with `@Audit(...)`; an undecorated handler costs nothing.

ROLE	Answer “who changed this record, when, and what were the old and new values” months later, in SQL, with an xlsx export.
NOT A SNAPSHOT	<code>changed</code> is a field <i>diff</i> . A no-op write is skipped entirely (<code>store.unchanged</code>).
SURVIVES DELETION	<code>entityLabel</code> stores the label <i>as it was</i> , so the row still reads correctly after a rename or a delete.
SCOPED READS	A non-super-admin's <code>actorId</code> filter is overwritten with their own id, <i>outside</i> the AND, so an ANY match mode cannot widen past them.
NEVER FAILS A REQUEST	The write already happened; losing its record must not fail the request that made it. A failed audit write is a logged <code>error</code> , nothing more.

The archive — the most carefully built lifecycle here

30 days hot in Postgres, then gzipped NDJSON at `audit/row-actions/YYYY/MM/DD.ndjson.gz`. Seven things it gets right, each guarding a specific failure:

1. Finds the **oldest day still owed** an archive, not just yesterday's boundary — so a run the worker missed is a delay, never a stranded day.
2. A **Redis lock per day**. Without it a second worker's page read lands after the first one's delete, uploads its short body over the complete object, verifies against itself, and the rest is gone.
3. Cursor-paginated, one gzip member per page, concatenated. The trailing newline in `toNdjson` is load-bearing — it stops a page boundary landing mid-record.
4. **Actor names resolved at write time**: an archive that needs a live database to say who did something is not an archive.
5. Upload, then **objectSize verified against byteLength**, and only then `deleteMany`. Select → write → verify → delete, the identical `[gte, lt)` window on every end.
6. A per-run day bound, with a warning when backlog remains.
7. Lock release is checked — an expired lock now belongs to a successor, which must keep it.

ImportLog + THE FILE

One row per import run: counts by outcome, the row-level errors as JSON, and `fileS3Key` — **the uploaded spreadsheet itself is kept**, so you can re-read exactly what was fed in, scoped to whoever may see the run.

Caddy's access log EBS

ROLE	The only record of requests the API <i>never saw</i> — TLS failures, 502s while a container restarted, a flood hitting a route that does not exist.
BOUNDED BY CADDY	JSON, one line per request, rolled at 100 MiB, gzipped, ten kept. Nothing in the repo configures this; it is Caddy's default.
OVERLAP	Deliberate. The app's line adds the actor and the request id, which Caddy cannot know; Caddy's covers what never reached the app.

BullMQ job records REDIS

The dead-letter queue. A completed job is evidence for **an hour** (1,000 kept) and then only occupies memory Redis may not evict. A job that exhausted its retries is kept **seven days** (500 kept), because nobody watches on the day it happens. It retains the *payload*, so a failed scoring job can be inspected and replayed.

5 · Every volume control in one place

WHAT COULD GROW	WHAT STOPS IT	WHERE
Per-request lines in production	<code>LOG_LEVEL ≥ log debug</code>	<code>logging.ts</code>
Container stdout on disk	<code>json-file</code> , 10 MB × 3 per container	<code>deploy/compose.yml</code>
Caddy access log	self-rolls, 100 MiB × 10 gzipped	Caddy default
Query logging	500 ms threshold; parameters excluded	<code>prisma.service.ts</code>
Prometheus series count	route <i>pattern</i> , never URL	<code>metrics.interceptor.ts</code>
A logged body	2 KB / 6 levels / 20 items	<code>redact.ts</code>
Completed BullMQ jobs	1 hour / 1,000	<code>queue/queues.ts</code>
Dead-letter jobs	7 days / 500	<code>queue/queues.ts</code>
<code>RowActionLog</code> table size	30 days → S3 → deleted	<code>audit-archive.ts</code>
Notification deliveries	prune processor + bounded pages	<code>notification-prune.processor.ts</code>

6 · What is still missing

Ranked, with the fix

#	GAP	FIX
1	Nothing ships off the box. Every stdout line and the whole access log live on Box A's EBS and are deleted when they roll. After the caps this is no longer a disk risk — it is a <i>retention</i> risk: a 5xx body from three days ago is already gone.	A Grafana Alloy container tailing the Docker socket into Loki (same free tier as the metrics), or switch the driver to <code>awslogs</code> with <code>mode: non-blocking</code> . Then move Caddy's log to stdout too — but not before , or you cut its retention from 1 GB to 30 MB.
2	Grafana Cloud is not actually receiving anything yet. The API produces <code>/metrics</code> and the log lines; nothing scrapes or ships them, so every signal in this document currently dies on Box A.	Point Grafana Cloud's hosted Prometheus at <code>/metrics</code> with the <code>METRICS_TOKEN</code> , then Alloy into Loki.
3	Caddy's log cannot be joined to the app's. The client only <i>reads</i> <code>x-request-id</code> off the response and never sends one, so the API mints a fresh id per request and Caddy's inbound-header log has no id to correlate on.	Have the typed client generate and send the header.
4	No client error reporting at all. <code>PageErrorBoundary</code> catches a page-level throw, shows an <code>EmptyState</code> with a reload, and <i>discards the error</i> . No <code>window.onerror</code> either. A student can white-screen and you will never know.	<code>componentDidCatch</code> → a small report endpoint on the API, which already knows how to log.
5	Domain lines are prose, not fields. Production now emits JSON <i>envelopes</i> , but the <code>message</code> inside is still a sentence — <code>Slow query (612ms): SELECT...</code> . Queryable by substring, not by value.	Only worth doing once a log store exists to query.
6	No client surfaces <code>requestId</code>, though every envelope carries it — so a student cannot quote the one token that would find their failure.	Show it in the failure <code>EmptyState</code> .

7 · Operator's cheat sheet

QUESTION

WHERE THE ANSWER IS

Is the API slow right now?

Grafana — `http_request_duration_seconds`. Never the logs.

QUESTION	WHERE THE ANSWER IS
Why did this one call fail?	stdout, the <code>error</code> line for its request id — stack and body.
What did the student actually send?	Same line. That is the only copy.
Is something new broken?	<code>sum by (bug) over level=error</code> , once logs ship.
Who changed this test's marks?	<code>RowActionLog</code> , via the admin audit screen. Not a log.
What was in that failed import?	<code>ImportLog.fileS3Key</code> — the original file.
Are sittings being scored?	<code>queue_depth</code> , <code>queue_oldest_waiting_seconds</code> , <code>scoring_backlog</code> .
Did a scoring job die?	BullMQ dead letters in Redis, 7 days, payload intact.
Did the request even reach us?	Caddy's access log on Box A.
Which calls is one student making?	<code>LOG_LEVEL=>=debug</code> , restart, then grep the actor. Put it back after.

Sources of record: `prisma/schema.prisma` for the audit tables, `docs/04-infrastructure.md` §9 for logs, metrics and alarms, and `.env.example` for `LOG_LEVEL` . This PDF is a reading aid; where it disagrees with those, they win.